

Se Solicita:

- Codificar el programa en lenguaje Assembler para 8086.

Diseño circuito secuencial que controle un mecanismo de elevación de un montacarga que se mueve entre tres puntos (planta baja, entre piso y planta alta). El circuito analiza una cadena de 2 bits introducida de manera serial y determina que acciones realizar y en qué lugar debería quedar el montacarga

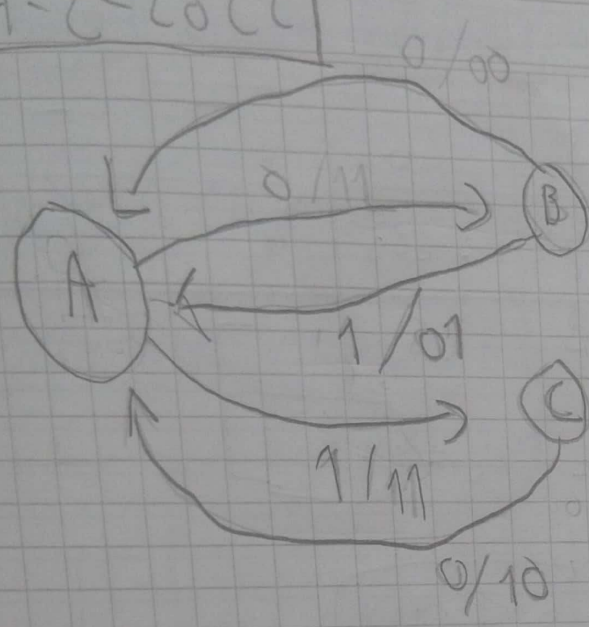
Se Pide:

- Defina el conjunto de entradas, salidas y estados que describe el comportamiento del sistema enunciado.
- Realice el diagrama y la tabla de estados.
- Diseñe el circuito con biestables tipo T.

Determine todos los valores para los siguientes esquemas de memoria, desarrollando los cálculos necesarios (no complete en la hoja de consignas).

[illegible]

Enal 24-2-2022



Se puede pasar de planta baja a planta alta sin

Necesidad de estar loco en el entre piso

Si no es directamente al ascensor, si quieres bajar del P1 al P4 no tienes que bajar directamente en P2 / P3 antes

Secuencia de Entrada:

- 00 = Entrepiso
- 01 = PA
- 10 = PB

Salida:

- 00 = Pasar a planta de entrepiso
- 01 = Pasar a planta de PA
- 10 = Pasar a planta de PB
- 11 = No sabe aún donde pasar aún

FINAL 24-2-22

1er METODO

include emu8086.inc

org 100h

.DATA

vector db 3h,0011_0001b,3h

vector_ASCII db 3h,0011_0010b,4h

.CODE

```
mov cx,2
Bucle_Vector:
push cx
mov si,cx
mov dl,vector[si]
mov cx,2
Bucle_Vector_ASCII:
mov si,cx
cmp dl,vector_ASCII[si]
JNE No_son_iguales
mov al,dl
CALL print_num
jmp proximo
No_son_iguales:
cmp cx,0
je proximo
Loop Bucle_Vector_ASCII
cmp cx,0
je Bucle_Vector_ASCII
proximo:
pop cx
cmp cx,0
je fin
loop Bucle_Vector
cmp cx,0
je Bucle_Vector
fin:
ret
```

DEFINE_PRINT_NUM
DEFINE_PRINT_NUM_UN\$

END

FINAL 24-2-22

2do METODO

org 100h

.DATA

vector db 3h,0011_0001b,3h

vector_ASCII db 3h,0011_0010b,4h

.CODE

```
push DS
POP ES
MOV CX,2
PUSH CX
Bucle_Vector:
LEA DI,vector_ASCII
cld
MOV bx,cx
mov al, vector[bx]
push cx
mov cx,3
REPNE SCASB
jz hay_coincidencia
continuar:
pop cx
cmp cx,0
je fin
loop Bucle_Vector
jmp Bucle_Vector
hay_coincidencia:
Call print_num
jmp continuar
fin:
ret
```

```
DEFINE_PRINT_NUM
DEFINE_PRINT_NUM_UN$
```

END

8086

Ver Metodo

Final 24-2-22

Name Final_24_2_22

Include 'emu8086.inc'

ORG 100h

DATA

Vector db ... ; este vendria a ser el Vector q' nos dan para evaluar

Vector_ASCII db 0011_0000b, 0011_0001b, 0011_0010b, 0011_0011b,
0011_0100b, 0011_0101b, 011_0110b, 011_0111b,
0011_1000b, 0011_1001b ; V los valores de nos en ASCII

~ = etiquetas
(lo pongo para
guiarme nomás)

CODE

importante

Mov CX, 9 ; Recordar q' los Arrays comienzan en 0

~ Bucle_Vector :

PUSH CX ; guardamos Valor de Bucle_Vector actual

Mov SI, CX

Mov DI, Vector[SI] ; usamos SI como indice / guardamos en DI (Xq' es db cada item del Array)

Mov CX, 9 ; para recorrer Siempre Vector_ASCII

~ Bucle_Vector_ASCII :

Mov SI, CX

Cmp DI, Vector_ASCII[SI]

JNE No_Son_iguales

Mov AL, DI

CALL Print_Num

Jmp Proximo

; Si ya me detecto q' Son = directamente Paso a evaluar el proximo Valor del Vector =

importante (Loop no evalúa el 0 llega hasta el 1)
↓

~ No-Son-iguales

CMP CX, 0

JE proximo

La razón de xq podemos estar así porq' en el Loop no se llega a evaluar nunca el 0 ya q' hacemos CX-1 y luego si CX=0 Sale del Bucle inmediatamente xende necesitamos de alguna manera evaluar el valor 0 del arreglo (Vector_Ascii[0])

Loop Bucle_Vector_ASCII

JMP Bucle_Vector_ASCII

al hacerse 0 y salir del Bucle, Forzoso a q' de nuevo retorne x ultima vez a la etiqueta del Bucle

~ proximo:

POP CX

CMP CX, 0

JE fin

Loop Bucle_Vector

JMP Bucle_Vector

; Saco de la pila el valor q' controla el bucle de "Vector" y lo pongo en CX

Lo mismo q' hice antes pero ahora con el Bucle de "Vector"

~ fin:

ret

define print_num

define print_num_vns

END

define print-num
define print-num-uns

END

2do Metodo

Name 'Final-24-2-22'

include 'Emu8086.inc'

ORG 100h

DATA

= 9º enter Metodo

CODE

inst
SB ←

```

push DS
pop ES
mov CX, 9
push CX

```

→ Para recorrer el 'vector'

~ Bucle_Vector:

```

LEA DI, Vector_ASCII ; para recorrer 'Vector_ASCII', ahí obtengo el offset
                        ; del Vector_ASCII y lo guardo en DI (ver inst SCASB)
Cld ; pone la flag DF=0 (es decir para q' recorra el vector de izquierda a
                        ; derecha) (DI+1)

```

```

mov BX, CX

```

```

mov al, Vector[BX]

```

```

push CX ; Resguardamos el CX de 'Vector'

```

```

mov CX, 10 ; para realizar el bucle de REPNE (q' recorre Vector_ASCII
            ; es 10 x q' en el ultimo valor de Vector_ASCII no lo evalua
            ; si es 9)
REPNE SCASB

```

```

JZ hay_Coincidencia

```

~ Continuar:

```

pop CX

```

```

cmp CX, 0 ; Para validar el 1er valor del vector (posición 0) ya q'
JE Fin ; el bucle_Vector no lo contempla
       ; = q' en el 1er Metodo

```

```

loop Bucle_Vector

```

```

jmp Bucle_Vector

```

~ hay Coincidencia:

```

call Print_Num

```

```

jmp Continuar

```

~ Fin:

```

ret

```

```

define Print_Num

```

```

define Print_Numbers

```

```

END

```